Java Abstraction

FOUNDATION GUIDE #6 OF 6 · JAVA OOP SERIES

The discipline of separating **contract from implementation** — and in FinTech, the difference between a system you can evolve and one you can only rewrite. Builds on Guides #1–#5: Classes & Objects, Encapsulation, Constructors, Inheritance, and Polymorphism.

*Developed by **Talenciaglobal***

What Is Abstraction?

Simple Intuition

A bank's risk engine doesn't care *how* fraud is detected — it just needs to ask: **"give me a fraud score for this transaction."** Maybe it's a rules engine, maybe it's an ML model, maybe it's a partner API. The engine talks to a `FraudDetector` — a **contract** that says "you must give me a score" — without knowing or caring which implementation runs underneath.

> **Abstraction is the discipline of separating *what something does* from *how it does it*.**

Formal Definition

Abstraction is the OOP principle of exposing **essential behavior (contracts)** while **hiding implementation details**. In Java, abstraction is expressed through two primary constructs:

- **Abstract classes** — partially implemented; can hold state and concrete methods; meant to be extended.
- **Interfaces** — pure contracts (originally); since Java 8 they carry default and static methods; since Java 9, private helper methods.

Core Business Value in FinTech

- **Swap implementations safely** — change KYC provider from Onfido to Veriff without touching business logic
- **Test in isolation** — mock LedgerRepository, FraudDetector, NotificationSender
- **Compliance boundaries** — separate "what must be reported" from "how it's transmitted"
- **Architectural layering** — domain knows nothing about HTTP, DB, or Kafka
- **Vendor independence** — abstract over cloud, message brokers, payment networks

Key Terminology

Mastering abstraction in Java requires fluency with the following terms — each representing a distinct mechanism in the abstraction toolkit.

Abstract Class

Class with `abstract` keyword; may have abstract methods (no body); **cannot be instantiated**

Interface

Pure contract; all methods implicitly public; can be implemented by **any class**

Default Method

Java 8+ interface method with a body — **backwards-compatible API evolution**

Functional Interface

Interface with exactly **one abstract method** — target for lambdas; annotated `@FunctionalInterface`

Sealed Interface

Java 17+ interface with **permitted implementers**; closed taxonomy enforced by compiler

Port / Adapter

Port = interface defined by domain; Adapter = implementation by infrastructure (Hexagonal Architecture)

Abstract Method

Declared without a body — subclasses must implement

Marker Interface

No methods — used as a type-level flag (`Serializable`, `Cloneable`)

Private Interface Method

Java 9+ helper method shared by default methods

Why Does Abstraction Exist? The FinTech Pain Points

Without abstraction, legacy FinTech systems become brittle monoliths. Industry data shows that **modernization projects in tightly-coupled banking systems average 3–5 years** and frequently exceed budget by 200%. Abstraction is the primary architectural remedy.

Pain	How Abstraction Solves It
Payment service tightly coupled to Razorpay; switching to Cashfree requires rewriting	`PaymentGateway` interface; Razorpay and Cashfree are interchangeable adapters
Cannot test business logic without a real database	`LedgerRepository` interface; tests use in-memory implementation
Compliance reporting logic duplicated across 4 regulators	Abstract `RegulatoryReporter` with shared formatting; subclasses for each regulator
Risk scoring needs to A/B test multiple algorithms	`RiskScorer` interface; runtime swap via configuration
Domain logic littered with `if (config.isProduction())`	Inject the right implementation; domain stays clean
New cloud provider deployment requires extensive code rewrites	Cloud-specific code behind `BlobStore` interface

⚠ **Counterfactual:** Your "payment service" imports `RazorpayClient` directly. Your "fraud module" imports `MaxMindGeoIPClient` directly. The day a vendor exits the market or a regulator changes mandates, you rewrite half the codebase. This is observable in many legacy systems and is the primary reason modernization projects take years.

Evolution & History of Abstraction in Java

Java's abstraction mechanisms have evolved significantly over three decades, with each major release addressing real-world pain points encountered by enterprise and FinTech developers.

1

Java 1.0 — 1996

Abstract classes and interfaces introduced as pure contracts

2

Java 5 — 2004

Annotations let frameworks "see" interface contracts (e.g., `@Repository`)

3

Java 8 — 2014

Default & static methods on interfaces; functional interfaces + lambdas (`Predicate`, `Function`, `Consumer`)

4

Java 9 — 2017

Private methods on interfaces; JPMS modules for package-level abstraction

5

Java 17 LTS — 2021

Sealed classes & interfaces — closed type hierarchies with compiler enforcement

6

Java 21 LTS — 2023

Pattern matching for switch + sealed interfaces → **exhaustive polymorphic dispatch**

> The 2026 canonical idiom: **Sealed interfaces + records + pattern matching** = the standard abstraction toolkit for serious FinTech systems. Hexagonal architecture is now the default architectural pattern in enterprise Java.

The Abstraction Spectrum

Java classes exist on a spectrum from most concrete to most abstract. Understanding where each construct sits guides architectural decisions.

1

Final Class

Most concrete. Cannot be subclassed. Designed for a single, fixed purpose.

2

Regular Class

Open for extension. Concrete implementation with no abstract methods.

3

Abstract Class

Partial implementation + abstract methods. Cannot be instantiated. Shared state allowed.

4

Interface

Most abstract. Pure contract (originally). Multiple inheritance. Default methods (Java 8+). Sealed (Java 17+).

Abstract Class — Key Anatomy

```
public abstract class Account {
  protected final AccountNumber number;
  protected Money balance;

  // Concrete shared method
  public final void debit(Money amount) {
    ensureActive();
    balance = balance.subtract(amount);
  }

  // Subclass must implement
  public abstract Money calculateMonthlyFee();
  protected abstract boolean canOverdraw();
}
```

Interface — Key Anatomy

```
public interface PaymentGateway {
  // Abstract — every implementer provides
  AuthResult authorize(Money amount,
  PaymentMethod method, IdempotencyKey key);

  // Default — backwards-compatible addition
  default boolean supportsPartialRefund() {
  return false;
  }

  // Static utility (Java 8+)
  static PaymentGateway noop() {
  return new NoopGateway();
  }
}
```

Abstract Class vs Interface — The Definitive Comparison

Choosing between an abstract class and an interface is one of the most consequential architectural decisions in Java OOP. The following table captures every relevant dimension.

Aspect	Abstract Class	Interface
Instantiation	Cannot be instantiated	Cannot be instantiated
Inheritance	One per subclass (single)	Many per implementer (multiple)
State (fields)	Yes — instance fields allowed	No instance fields (only `public static final` constants)
Constructors	Yes (called via `super(...)`)	No constructors
Method bodies	Yes — concrete + abstract	Default (Java 8+), static, private (Java 9+)
Access modifiers	Any (private, protected, public)	Abstract/default implicitly public; private (Java 9+)
Purpose	Share implementation + state across is-a relatives	Declare capabilities / contracts
Records can use	Cannot extend a class	Can implement interfaces
API evolution	Adding abstract method breaks subclasses	Default method adds without breaking
Sealed support	Sealed classes (Java 17+)	Sealed interfaces (Java 17+)

Industry rule of thumb: **If implementations share state and a partial workflow → abstract class. If you need a pluggable contract with multiple independent implementers → interface.** When in doubt, prefer interface — it imposes fewer constraints on implementers.

Sealed Interfaces — The Modern Closed-World Idiom

Introduced in Java 17 (LTS), sealed interfaces represent one of the most significant additions to Java's abstraction toolkit. They are particularly powerful in FinTech, where **closed business taxonomies** (payment methods, account states, regulatory events) must be exhaustively handled.

Sealed PaymentMethod — Full Example

```
public sealed interface PaymentMethod
    permits Card, Upi, NetBanking,
            Wallet, Bnpl, Emi { }

public record Card(String token,
    CardBrand brand)
    implements PaymentMethod { }
public record Upi(String vpa)
    implements PaymentMethod { }
public record NetBanking(String bankCode)
    implements PaymentMethod { }
public record Wallet(String walletId,
    String provider)
    implements PaymentMethod { }
public record Bnpl(String providerId,
    int tenureMonths)
    implements PaymentMethod { }
public record Emi(String issuerId,
    int tenureMonths,
    BigDecimal interestRate)
    implements PaymentMethod { }

// Exhaustive switch — no default needed:
public Money calculateFee(
    Money amount, PaymentMethod method) {
  return switch (method) {
    case Card c ->
        amount.multiply(new BigDecimal("0.015"));
    case Upi u ->
        Money.zero(amount.currency());
    case NetBanking n -> Money.inr("10");
    case Wallet w ->
        amount.multiply(new BigDecimal("0.010"));
    case Bnpl b ->
        amount.multiply(new BigDecimal("0.025"));
    case Emi e ->
        amount.multiply(new BigDecimal("0.012"));
  };
}
```

Why Sealed Wins in FinTech

Compiler Enforcement

A rogue `class Crypto implements PaymentMethod` won't compile — not in `permits`

Exhaustive Dispatch

Every switch over `PaymentMethod` is forced to handle every case — compiler-enforced completeness audit

Zero Inheritance Ceremony

Records keep each method type tiny and immutable — no complex class hierarchies

Production Safety

Eliminated production incidents from incomplete dispatch; new payment-method proposals go through product review *before* the code compiles

Functional Interfaces & the java.util.function Toolkit

Functional interfaces — those with exactly one abstract method — are the bridge between Java's type system and lambda expressions. In FinTech, they power event handlers, validators, retry policies, and transformation pipelines. **Over 43 functional interfaces** ship in `java.util.function`; the most critical for FinTech are below.

Interface	Signature	FinTech Use
`Function<T, R>`	`R apply(T)`	`Function<RawTxn, NormalizedTxn>` for ETL pipelines
`Predicate<T>`	`boolean test(T)`	`Predicate<Customer> isKycVerified`
`Consumer<T>`	`void accept(T)`	`Consumer<PaymentEvent> publish`
`Supplier<T>`	`T get()`	`Supplier<Instant> clock` (testable time)
`BiFunction<T,U,R>`	`R apply(T, U)`	`BiFunction<Account,Money,Account> applyFee`
`UnaryOperator<T>`	`T apply(T)`	`UnaryOperator<Money> applyDiscount`
`BinaryOperator<T>`	`T apply(T, T)`	`BinaryOperator<Money>` for summation

Functional Interface + Lambda — Retry Policy Example

```
@FunctionalInterface
public interface RetryPolicy {
 Optional<Duration> nextDelay(int attemptNumber, Throwable lastError);

 static RetryPolicy fixed(Duration delay, int maxAttempts) {
 return (attempt, err) -> attempt < maxAttempts
 ? Optional.of(delay) : Optional.empty();
 }
 static RetryPolicy exponential(Duration base, int maxAttempts) {
 return (attempt, err) -> attempt < maxAttempts
 ? Optional.of(base.multipliedBy(1L << attempt)) : Optional.empty();
 }
}

// Custom policy — lambda with business logic:
RetryPolicy custom = (attempt, err) -> {
 if (err instanceof RateLimitException rle) return Optional.of(rle.retryAfter());
 if (attempt < 3) return Optional.of(Duration.ofSeconds(1));
 return Optional.empty();
};
```

Use Case 1 — Ports & Adapters (Hexagonal Architecture)

REAL-WORLD FINTECH

Context: A lending platform's domain code was tangled with Postgres specifics, S3 specifics, and Razorpay specifics. Testing required real infrastructure. Switching from Razorpay to Cashfree was a **6-month project**. After applying Hexagonal Architecture with interface-based ports, the same gateway swap took **3 weeks**.

Domain Layer — Pure Ports (No Infrastructure)

```
package com.bank.loans.domain;

public interface LoanRepository { // PORT
  Optional<Loan> findById(LoanId id);
  void save(Loan loan);
  List<Loan> findOverdue(LocalDate asOf);
}
public interface PaymentGateway { // PORT
  AuthResult authorize(Money amount,
    PaymentMethod method, IdempotencyKey key);
}
public interface DocumentStore { // PORT
  String store(byte[] content, String type);
  byte[] retrieve(String docId);
}

// Domain service — ZERO infrastructure imports:
public final class LoanDisbursementService {
  private final LoanRepository loanRepo;
  private final PaymentGateway gateway;
  private final DocumentStore docStore;

  public DisbursementResult disburse(
      LoanId loanId, BorrowerAccount target) {
    Loan loan = loanRepo.findById(loanId)
        .orElseThrow();
    AuthResult auth = gateway.authorize(
        loan.amount(), target.method(),
        loan.idemKey());
    loan.markDisbursed(auth.transactionId());
    loanRepo.save(loan);
    return DisbursementResult.of(loan, auth);
  }
}
```

Infrastructure Layer — Adapters

```
package com.bank.loans.infrastructure;

public final class PostgresLoanRepository
    implements LoanRepository { // ADAPTER
  private final JdbcTemplate jdbc;
  @Override
  public Optional<Loan> findById(LoanId id) {
    /* SQL */
  }
  @Override
  public void save(Loan loan) { /* SQL */ }
  @Override
  public List<Loan> findOverdue(
      LocalDate asOf) { /* SQL */ }
}

public final class RazorpayGateway
    implements PaymentGateway { /* HTTP */ }
public final class CashfreeGateway
    implements PaymentGateway { /* HTTP */ }
public final class S3DocumentStore
    implements DocumentStore { /* S3 */ }
```

Test Setup — No Infrastructure Needed

```
var service = new LoanDisbursementService(
  new InMemoryLoanRepo(),
  new FakeGateway(),
  new InMemoryDocStore()
);
```

Outcome: Domain test suite runs in seconds. Gateway swap: 3 weeks (one new adapter + config), not 6 months. Regulatory audit: point to domain package — no SQL, no HTTP noise.

Use Case 2 — Abstract Class for Shared Regulatory Reporting

TEMPLATE METHOD PATTERN

Context: A bank submits regulatory reports to RBI (CRILC), CIBIL, Experian, and TransUnion. Each requires different file formats, schedules, and submission endpoints — but the **shape of the process is identical**: query data, format records, sign, submit, log, retry. Adding a new regulator previously required duplicating ~400 lines of boilerplate. With the abstract class approach, a new regulator requires **~80 lines**.

Abstract Base — Template Method

```
public abstract class RegulatoryReporter {
  protected final ClockService clock;
  protected final AuditLog audit;
  protected final Signer signer;

  // TEMPLATE METHOD — invariant flow
  public final SubmissionResult submit(
      LocalDate reportingDate) {
    audit.record(name() + " started");
    try {
      List<? extends ReportRecord> records =
          collectRecords(reportingDate);
      if (records.isEmpty())
          return SubmissionResult.empty();
      byte[] payload = format(records);
      byte[] signed = signer.sign(payload);
      SubmissionResult result =
          transmit(signed);
      audit.record(name() + " submitted");
      return result;
    } catch (Exception e) {
      audit.record(name() + " FAILED");
      throw new ReportingException(name(), e);
    }
  }

  // Hooks — subclasses implement:
  protected abstract String name();
  protected abstract List<? extends
      ReportRecord> collectRecords(
          LocalDate date);
  protected abstract byte[] format(
      List<? extends ReportRecord> records);
  protected abstract SubmissionResult
      transmit(byte[] signedPayload);
}
```

CIBIL Subclass — ~80 Lines

```
public final class CibilReporter
    extends RegulatoryReporter {
  private final CustomerRepo customers;
  private final SftpClient sftp;

  @Override
  protected String name() { return "CIBIL"; }

  @Override
  protected List<? extends ReportRecord>
      collectRecords(LocalDate d) {
    return customers.findChangesSince(d)
        .stream()
        .map(CibilRecord::from)
        .toList();
  }

  @Override
  protected byte[] format(
      List<? extends ReportRecord> records) {
    /* CIBIL fixed-width format */
  }

  @Override
  protected SubmissionResult transmit(
      byte[] payload) {
    return sftp.upload(
        "cibil/" + LocalDate.now() + ".dat",
        payload);
  }
}
```

Why abstract class (not interface): Shared workflow lives in the parent. Shared state (clock, audit, signer) is common. The `final` template method ensures audit + signing are always called — subclasses cannot accidentally bypass compliance controls.

Default Methods — Safe API Evolution

One of the most practically impactful features introduced in Java 8, default methods solve a fundamental problem: **how do you add a method to a widely-implemented interface without breaking every existing implementer?** In large FinTech codebases with dozens of interface implementations, this was previously a breaking change requiring coordinated updates across teams.

```
public interface FraudDetector {
  int score(Transaction txn); // v1

  // v2 — adds explanation WITHOUT breaking
  // existing implementers:
  default FraudExplanation explain(
      Transaction txn) {
    return new FraudExplanation(
        score(txn),
        List.of("no detail available"));
  }
}

// Existing implementations keep compiling:
public final class SimpleRuleDetector
    implements FraudDetector {
  @Override
  public int score(Transaction txn) { /* ... */ }
  // explain() inherits the default — no change
}

// New implementations can override for richer behavior:
public final class MLFraudDetector
    implements FraudDetector {
  @Override
  public int score(Transaction txn) {
    /* ML inference */
  }
  @Override
  public FraudExplanation explain(
      Transaction txn) {
    return new FraudExplanation(
        score(txn),
        this.featureImportance(txn));
  }
}
```

Backwards Compatible

Existing implementers continue to compile and run without modification

Sensible Fallback

Default provides a reasonable baseline; sophisticated implementers override

Caution

Don't add business logic to default methods — they become hard-to-override surprises. Use for *sensible fallbacks* only

Interface Segregation Principle (ISP)

The Interface Segregation Principle — one of the SOLID principles — states: **don't force consumers to depend on methods they don't use.** Fat interfaces are a common anti-pattern in FinTech systems, where account services accumulate dozens of methods over years of feature additions.

❌ Fat Interface — ISP Violation

```
public interface AccountService {
  void debit(Money amount);
  void credit(Money amount);
  Money balance();
  void freeze();
  void unfreeze();
  void changeKyc(KycDocument doc);
  void approveLoan(LoanApplication app);
  void generateStatement(
     LocalDate from, LocalDate to);
}
```

Every consumer — even one that only needs to read a balance — must depend on `approveLoan` and `changeKyc`. Tests must mock all methods. Changes to any method affect all consumers.

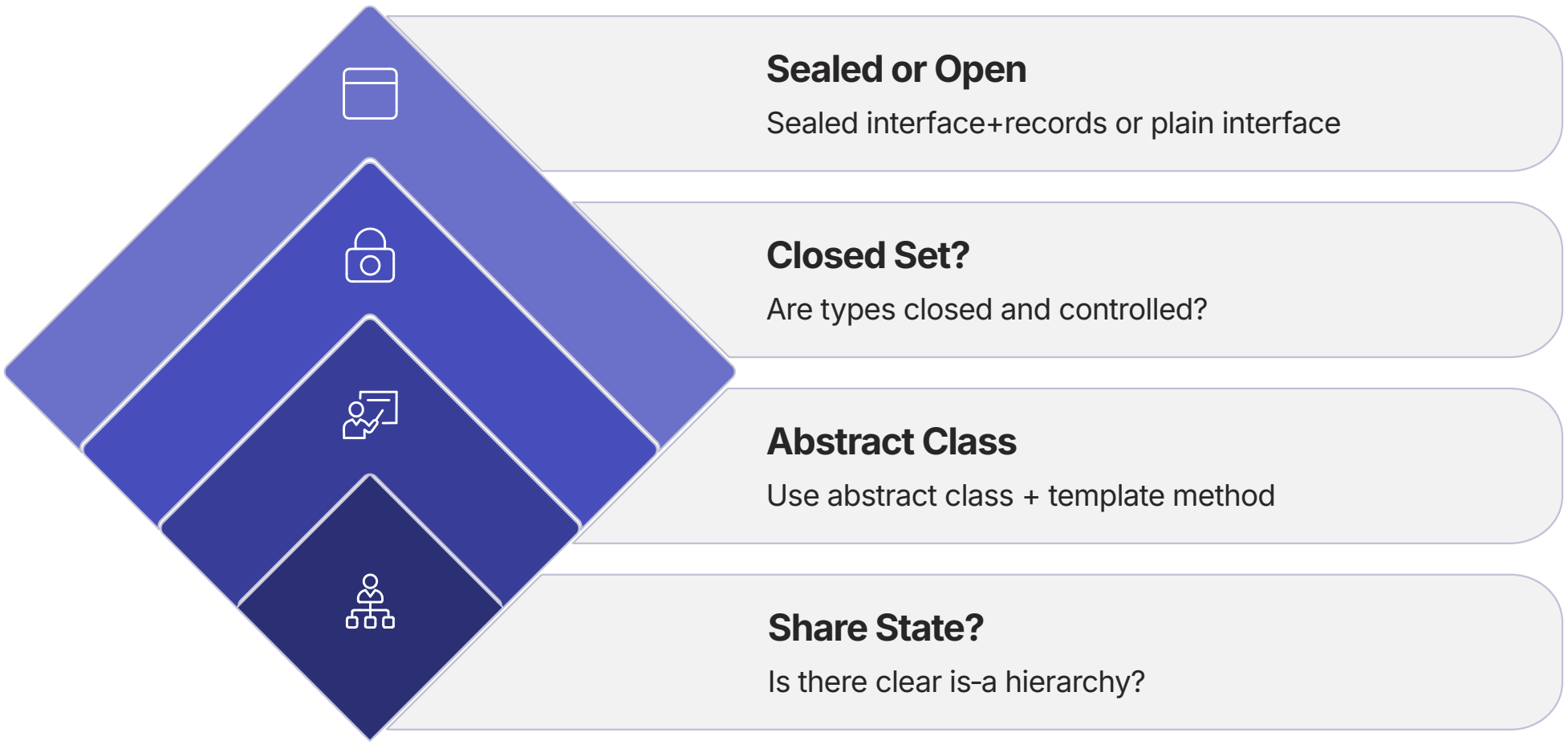
✅ Segregated Interfaces — ISP Compliant

```
public interface AccountTransactionPort {
  void debit(Money amount);
  void credit(Money amount);
  Money balance();
}
public interface AccountLifecyclePort {
  void freeze();
  void unfreeze();
}
public interface KycPort {
  void changeKyc(KycDocument doc);
}
public interface LoanApprovalPort {
  void approveLoan(LoanApplication app);
}
public interface StatementGenerationPort {
  /* ... */
}
```

Consumers and tests depend only on what they need. A fraud detection service depends only on `AccountTransactionPort` — it never sees loan approval methods.

Decision Framework — When to Choose What

The following framework guides the most consequential abstraction decision in Java architecture. Apply it at every module boundary, every service interface, and every domain type definition.



This three-way decision covers the vast majority of real-world abstraction choices. The key insight is that **sealed interfaces occupy a middle ground** — more constrained than open interfaces, but more flexible than abstract classes — making them ideal for FinTech's closed business taxonomies.

Abstract Class

Shared workflow + state. Template Method pattern. Genuine is-a hierarchy. Example: `RegulatoryReporter`, `Account`

Sealed Interface + Records

Closed taxonomy. Exhaustive dispatch. Compiler-enforced completeness. Example: `PaymentMethod`, `PaymentEvent`

Open Interface

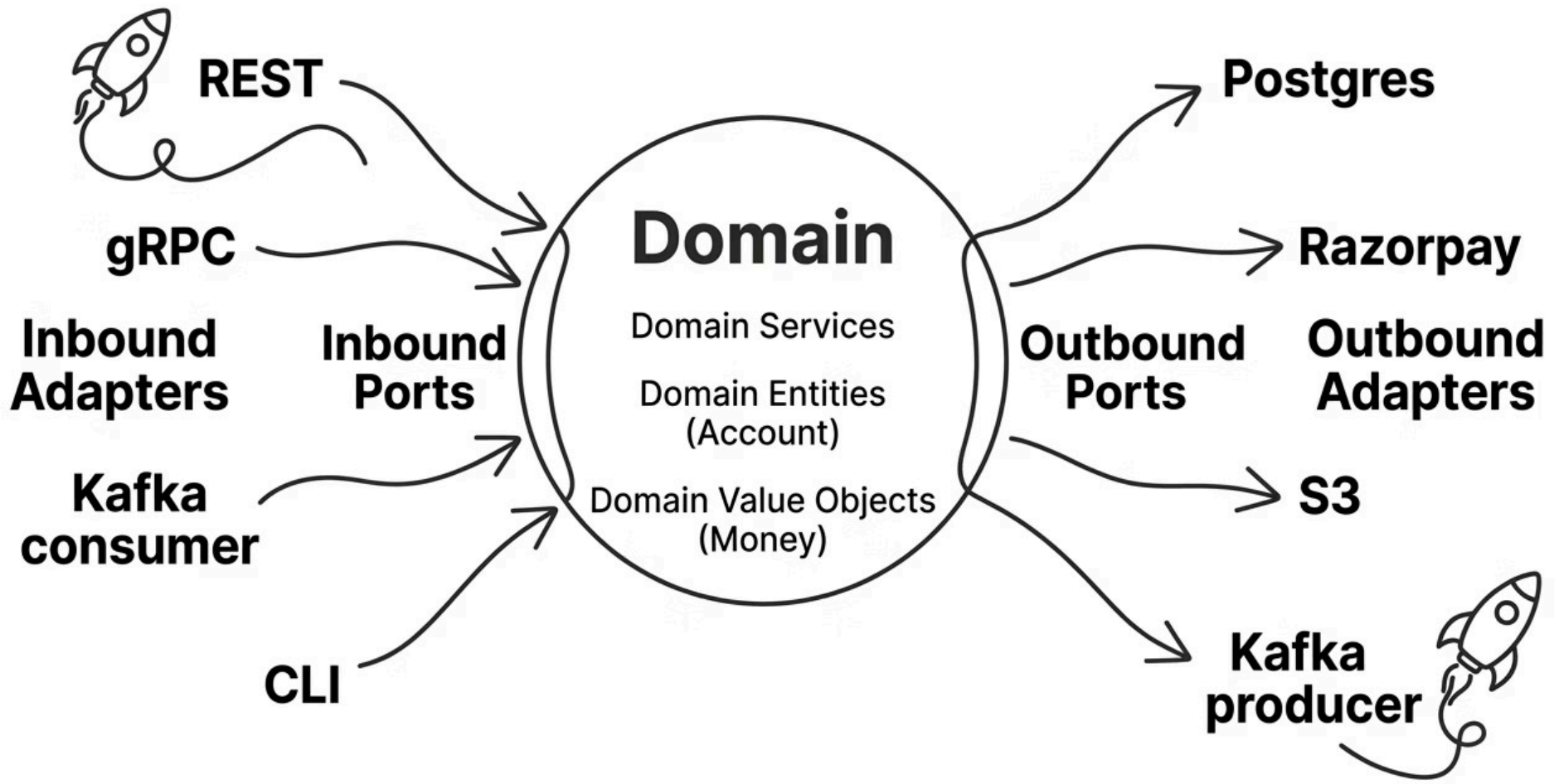
Pluggable contracts. Ports & adapters. Multiple independent implementers. Example: `PaymentGateway`, `LoanRepository`

Functional Interface

Single-method pluggable behavior. Lambda-friendly. Example: `RetryPolicy`, `FraudScorer`, `Predicate<Transaction>`

Hexagonal Architecture — The FinTech Standard

Hexagonal Architecture (Ports & Adapters, Cockburn 2005) is the single most impactful abstraction discipline in FinTech. It uses interfaces to create an **explicit boundary between domain logic and infrastructure**. Industry surveys indicate that teams adopting hexagonal architecture report **60–70% reduction in time-to-test** for business logic and significantly faster vendor swap cycles.



Inbound Ports (Use Cases)

Interfaces called by adapters (HTTP, queue, CLI). Define what the application *can do* from the outside world's perspective. The domain defines these — adapters call them.

Outbound Ports (Dependencies)

Interfaces the domain *needs* (repository, gateway, document store). The domain defines these — infrastructure adapts to them. This is the key inversion: infrastructure serves the domain.

> **The golden rule:** Define interfaces from the consumer's perspective, in the domain layer. Infrastructure adapts to the domain, not vice versa. Domain code has zero infrastructure imports — pure business logic, auditable in isolation.

Advantages, Disadvantages & Anti-Patterns

Abstract Class

✅ Advantages

- Can carry state and concrete behavior
- Template Method for invariant workflows
- Protected helpers shared across subclasses
- Single, focused taxonomy under a base class

⚠️ Disadvantages

- Single inheritance — child can't extend anything else
- Tight coupling to parent's implementation (fragile base)
- Adding abstract methods breaks subclasses

Interface

✅ Advantages

- Multiple implementation — satisfy many contracts
- Pure contracts decouple consumers from implementations
- Default methods enable safe API evolution
- Records can implement interfaces
- Sealed interfaces give closed-world exhaustiveness

⚠️ Disadvantages

- No instance state
- All non-private methods implicitly public
- Conflicting default methods require explicit resolution

Critical Anti-Patterns to Avoid

Anti-Pattern	Why It's Bad	Fix
Interface with one implementation + no swap horizon	Unnecessary indirection	Remove the interface; refactor when need arises
Interface with 30+ methods	Fat interface; ISP violation	Split by capability
Concrete class extended for code reuse only	Inheritance abuse	Composition or extract interface
Returning concrete types from public APIs	Couples callers to impl	Return interface (`Map`, not `HashMap`)
Default methods carrying business logic	Hard to override; surprises	Move logic to abstract class or service

Performance & Scalability Considerations

A common concern among engineers new to abstraction is runtime overhead. Modern JVM evidence is clear: **interface dispatch overhead is negligible in the vast majority of FinTech workloads.** The real performance story is about design quality, not dispatch cost.

Dispatch Cost

Interface method calls use an **itable** (similar to vtable). Modern JVMs (HotSpot, GraalVM) make interface vs class dispatch essentially equivalent in hot paths. Sealed interfaces help the JIT — closed world enables more frequent devirtualization.

Memory Footprint

Abstract classes have instance fields → memory footprint per instance. Interfaces add no instance fields → implementing additional interfaces is **free** in terms of memory. Records implementing interfaces are particularly efficient.

Common Pitfalls

Megamorphic interface calls in tight loops can inhibit JIT inlining. **Reflective interface dispatch** in frameworks causes slow startup. For ultra-hot paths: consider sealed interfaces for closed taxonomies to enable devirtualization.

Design Scalability

Abstractions scale design, not runtime. Their value is in maintainability and team velocity. Decoupled deployment, independent testing, team boundaries, and vendor/regulatory change tolerance are the real scalability wins.

⚠ **The real performance risk:** Wrong abstractions cost dearly. A leaky abstraction — one that exposes implementation details — propagates change throughout the system, costing engineering time that dwarfs any dispatch overhead. Design correctly first; optimize only with profiler evidence.

Security, Compliance & Module-Level Abstraction

Security & Compliance via Abstraction

Risk	Mitigation
Direct vendor SDK dependency leaks credentials	Adapter pattern; vendor SDK confined to one class
Sensitive flow logic spread across system	Centralize in abstract class with `final` template method
Untrusted code provides malicious implementer	Sealed interfaces; security-critical types `final`
Audit trail incomplete because flow varies per impl	Template method ensures audit always called
Cross-cutting concerns reimplemented inconsistently	Default methods or abstract base ensure uniform behavior
Module-level secret leakage	JPMS with strict exports; secrets in unexported packages

JPMS Module-Level Abstraction

```
// Domain module — exposes only ports
module com.bank.loans.domain {
  exports com.bank.loans.domain.api;
  exports com.bank.loans.domain.ports;
  // internals not exported
}

// Infrastructure — depends on domain ports
module com.bank.loans.infrastructure {
  requires com.bank.loans.domain;
  requires java.net.http;
  requires org.postgresql.jdbc;
  exports com.bank.loans.infrastructure.config;
  // adapters not exported
}

// App — wires it all together
module com.bank.loans.app {
  requires com.bank.loans.domain;
  requires com.bank.loans.infrastructure;
  requires spring.context;
}
```

ⓘ JPMS enforces that infrastructure imports go through domain ports at the **compiler level**. Domain cannot accidentally import infrastructure — the module system rejects it.

The Architect's Cheat Sheet — Abstraction Mechanisms

A comprehensive reference for selecting the right abstraction mechanism for every FinTech scenario. Apply this at every architectural decision point.

Mechanism	Best For	FinTech Examples
Abstract class + Template Method	Shared workflow with varying steps	`RegulatoryReporter`, `BatchJob`, `Account`
Interface (open)	Pluggable contracts; ports & adapters	`PaymentGateway`, `LoanRepository`, `DocumentStore`
Sealed interface + records	Closed taxonomies, exhaustive dispatch	`PaymentMethod`, `PaymentEvent`, `KycDocument`
Functional interface + lambda	Pluggable single-method behavior	`RetryPolicy`, `FraudScorer`, `Predicate<Transaction>`
Marker interface	Type-level flag for cross-cutting dispatch	`RegulatoryReportable`, `AuditSensitive`
Annotation + reflection	Cross-cutting metadata, framework hooks	`@Transactional`, `@AuditLogged`
Module (JPMS)	Coarse-grained encapsulation; team boundaries	domain, infrastructure, app modules

Different teams write different implementations

→ **Interface**

All implementations share half the code

→ **Abstract class**

Finite, controlled set of types

→ **Sealed interface + records**

Plug in a single function

→ **Functional interface**

Add a method to a long-deployed interface

→ **Default method**

Strong encapsulation across JARs

→ **JPMS module**

Final Takeaway & Series Recap

> **"In FinTech, the right abstraction lets you swap a payment gateway, replace a database, satisfy a new regulator, and onboard a new vendor — all without touching business logic. The wrong abstraction lets none of that. Choose carefully; you'll live with the choice for a decade."**

The Default Mental Model for 2026

01

Boundaries between domain and infrastructure?

→ Interfaces (ports/adapters). Domain defines; infrastructure adapts.

02

Shared workflow with varying steps?

→ Abstract class + template method. Final method enforces invariant flow.

03

Closed set of business types?

→ Sealed interface + records + pattern matching. Compiler-enforced completeness.

04

Pluggable single-method behavior?

→ Functional interface + lambdas. Lambda-friendly, composable.

05

Strong encapsulation across packages/JARs?

→ JPMS modules. Compiler-enforced boundaries.

Java OOP Foundations Series — Complete

#	Topic	Key Idea
1	**Classes & Objects**	The blueprint and the instance; the canonical FinTech domain model
2	**Encapsulation**	Hide state; expose behavior; enforce invariants
3	**Constructors**	Establish invariants atomically at birth
4	**Inheritance**	Genuine is-a relationships; favor composition for the rest
5	**Polymorphism**	One call, many forms — at compile time, runtime, and via patterns
6	**Abstraction**	Contracts that outlive implementations

Together, applied with sealed types + records + pattern matching, these six principles form the **modern Java OOP idiom** for FinTech systems in 2026: type-safe, audit-friendly, evolution-ready, and architect-grade. **Developed by Talenciaglobal.**